Gated Community Management System - Phase 2 API

This repository contains the backend API implementation for Phase 2 of the Gated Community Management System.

Tech Stack

- **Python 3.10+
- **Flask** (Micro-framework for API routing)
- **mysql-connector-python** (Raw SQL execution; NO ORM used)
- **PyJWT & bcrypt** (Authentication & Security)

Setup & Running Locally

1. Clone the repository and enter the directory.

2. Set up Virtual Environment:
bash
python -m venv venv

3. Install Dependencies:
bash
pip install -r backend/requirements.txt

4. Database Configuration:
- Ensure MySQL is running locally.
- Execute `schema.sql` and `seed.sql` from Phase 1 to set up the database and mock data.
- Copy `.env.example` to `.env` and fill in your database credentials:
bash
cp .env.example .env

5. Start the API Server:
```bash
python -m backend.app
```
The server will run on `http://localhost:5000`.

Testing API Endpoints

You can use Tools like **Postman**, **cURL**, or import the included `swagger.yaml` into [Swagger Editor](https://editor.swagger.io) to interact with the API.

1. Register a new Resident
bash
curl -X POST http://localhost:5000/api/v1/auth/register \

```
-H Content-Type: application/json \
-d '{"username": "new_resident", "password": "securepassword", "email": "new@email.com", "unit_id": 1, "residency_status": "Owner"}'
```

2. Login to get JWT

bash

```
curl -X POST http://localhost:5000/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username": "new_resident", "password": "securepassword"}'
```

3. Access Protected Route (e.g. Booking an Amenity)

Requires JWT from Login step placed in Authorization header

bash

```
curl -X POST http://localhost:5000/api/v1/amenities/book \
-H "Authorization: Bearer <YOUR_JWT_TOKEN>" \
-H "Content-Type: application/json" \
-d '{"amenity_id": 1, "booking_date": "2026-05-01", "start_time": "10:00:00", "end_time": "12:00:00"}'
```